ICT133

Timed Online Assignment – January Semester 2021

Structured Programming

Thursday, 20 May 2021 **1:00 pm – 3:30 pm**

Time allowed: 2.5 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **FOUR (4)** questions and comprises **NINE (9)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: TOA Course Code, Your Name, and SUSS PI No. Name your submission file as TOACourseCode_FullName_StudentPI. Use underscore and NOT Space. *Example: TOAICT133_RaphaelLee_T1923161 (omit D/O, S/O).*

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .doc or .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the Word file provided it does not exceed the file size limit of 500MB. Images of handwritten answers will not be marked.

7. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

8. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

Answer all questions. (Total 100 marks)

You can use ONLY built-in functions and packages covered in the module, e.g., math, random. You can use ONLY built-in types. Do not write classes.

Question 1

Express a sequence of statements based on computational logic.

(a) Write statements for the following tasks:

(i) Given a 2-digit whole number, `n`, display the message `Valid` if `n` contains the digit 1. For example, 10, 11 and 31 are valid but 33, 34 and 35 are not. Do not convert `n` into a string.

(3 marks)

(ii) Given that string `s` has exactly 3 characters. Display the message `Group A` if characters in the first, middle and last positions are the same. For example, if `s` is ***'222'***, display `Group A`.
Display message `Group B` if the first and last characters are the same but the middle character is different. For example, if `s` is ***'232'***, display `Group B`.
Display message `Group C` if the first and last characters are not the same. For example, if `s` is ***'223'*** or ***'213'***, display `Group C`.

(7 marks)

(iii) Given a list of numbers, `nList` such as [1, 3, 2]. If the sum of digits in the even position is even, display the message `Sum of digits in even positions is even.` Otherwise, display the message `Sum of digits in even positions is odd`.
Do not use loop/repetition structure. Instead, use the slice operator and the built-in function `sum`.

(3 marks)

(b) Given a string, `s` that contains a sequence of digits.

(i) Write a code segment that picks each digit in the string in reverse order, reduces each digit by 1 and displays the resulting number on separate lines. The code segment should produce the output for a given string `'1204'`:

```
3
-1
1
0
```

(5 marks)

(ii) Write a code segment that prints the message `True` if the sum of each pair of adjacent digits does not exceed 12. Print the message `False` otherwise. For illustration, suppose the string `s` has 5 digits: $d_0d_1d_2d_3d_4$.
If $d_0 + d_1$, $d_1 + d_2$, $d_2 + d_3$, $d_3 + d_4$ do not exceed 12, the code segment prints `True`. Thus, for the string `'48531'`, the code segment prints the message `False` because the sum of pair of adjacent digits `85`, or `8+5` exceeds 12.
(7 marks)

Question 2

This question assumes that there are two collections: One collection stores 20 product names and prices from store X and the other collection stores the same product names but with prices from store Y.

You are to propose structure for the collection for tasks 1 to 3 in part a), and then based on your proposed collection structure in part a), propose the control structure for part b).

<u>Task 1</u>
Get the average price difference for all 20 products offered in store X and store Y. If the average price from store X is cheaper, the average difference will be negative.

<u>Task 2</u>
Display the product name and prices of products that are cheaper in store X than in store Y.

<u>Task 3</u>
For one given product name, display its prices from store X and store Y.

(a) Use your understanding of how data structures store and process information.

- Propose a structure for the collection for each task. The structure may be different for a different task.
- Justify the structure you propose for each task based on the required operations for each task.

(12 marks)

(b) Use your understanding of the building blocks of computers and programs, and determine which Python control structures apply to tasks 1 to 3:

- `if/if-else` selection and
- `for/while` repetition.

For each task, explain your choices of control structures as well as how the control structures are applied in the logic for performing the task, based on the collection structures proposed in Q2(a).

Note that more than one control structures may be required for a task, but you are restricted to using each control structure to a maximum of 2 tasks of tasks 1 to 3. Using built-in functions such as sum does not add to the count of each of the two listed control structures used. Also, note that list comprehension is considered as a repetition structure. Furthermore, a list comprehension that includes `if` is considered as a repetition structure with a selection structure.

(13 marks)

Question 3

Employ structured programming principles to develop a program. **Do NOT use global variables.**

(a) Implement a function, **`roll`** that rolls a dice until 3 different dice values are obtained. The function returns the sum of the three rolls with different values.

(6 marks)

(b) Implement a function, **`playGame`** which has two parameters:

- `playerList`, a list of player names `and`
- `maxLimit`, a number

The function calls the function **`roll`** for each player, and identifies the highest sum of rolls, `highest`. The identified `highest` should not exceed `maxLimit`. The function returns a list of player names whose sum of rolls is the same value as `highest`.

(10 marks)

(c) Write a function, **`main`** which has a list of 5 player names: `A`, `B`, `C`, `D` and `E`, and `maxLimit` set to 12. The function repeatedly allows the user to select one menu choice:

```
Menu
1. Change max limit for sum of 3 rolls
2. Play Game
0. Exit
```

Option 1 allows the user to update `maxLimit` to a new value no less than 10. Option 2 calls the function **`playGame`** and displays the winner(s).

You may introduce one or more functions to modularise the function, **`main`**.

<u>Sample run (input underlined):</u>

```
Menu
1. Change max limit for sum of 3 rolls
2. Play Game
0. Exit
Enter choice: 4
Invalid choice
1. Change max limit for sum of 3 rolls
2. Play Game
0. Exit
Enter choice: 2
A, E
Menu
1. Change max limit for sum of 3 rolls
2. Play Game
0. Exit
Enter choice: 2
B
Menu
1. Change max limit for sum of 3 rolls
2. Play Game
0. Exit
Enter choice: 1
Enter new max limit: 8
Please enter a number that is at least 10
Enter new max limit: 15
Menu
1. Change max limit for sum of 3 rolls
2. Play Game
0. Exit
Enter choice: 2
D
Menu
1. Change max limit for sum of 3 rolls
2. Play Game
0. Exit
Enter choice: 0
Application ends
```

(9 marks)

Question 4

Solve computational problems using structured programming. **Write a structured program** to manage a product list as a dictionary and to record the product list in a text file, products.txt.

(a) The text file, products.txt contains lines of product data: code, name and price, separated by comma (,). Some lines are shown here:

```
A020, 8, 137 Degree Real Almond Milk
A121, 4.95, UFC Velvet Almond Milk
B123, 3.15, Wholemeal Bread
B138, 5.95, Fruit and Nut Bread
```

(i) Write a function to read product data from the file, products.txt into a dictionary in this format, and returns the dictionary:

```
{'A020': [8.0, '137 Degree Real Almond Milk'],
'A121': [4.95, 'UFC Velvet Almond Milk'],
'B123': [3.15, 'Wholemeal Bread'],
'B138': [5.95, 'Fruit and Nut Bread']}
```

(ii) Write a function that takes the dictionary of product data, and save the content in the dictionary back to the file, products.txt, in the same format - each line in the file records one product data: code, name and price, separated by comma (,). The lines in product.txt are sorted in ascending order product price. An example content of products.txt with lines sorted in ascending order product price is shown here:

```
B123,3.15,Wholemeal Bread
A121,4.95,UFC Velvet Almond Milk
B138,5.95,Fruit and Nut Bread
A020,8.0,137 Degree Real Almond Milk
```

(10 marks)

(b) Write a program using functions to organise your code. The program does the following tasks:

(i) The program calls the function in Q4(a)(i) to read the file, product.txt and get a dictionary with the content of the file. The program displays the dictionary.

(ii) The program allows the user to repeatedly enter a product code.

- To terminate, the user enters an empty string by simply hitting the ENTER key. The program calls the function in part a) ii) to save the dictionary content to the file and the program terminates.
- To update the price of a product, the user enters an existing product code. The program prompts for the new price. If the new price is not a positive number, it displays an error message 'Price must be a positive value'. Otherwise, the program updates the price of the existing product and displays the dictionary.
- To add a new product, the user enters a new product code that does not exist in the dictionary. The program prompts for a price. If the price is not a positive number, it displays an error message 'Price must be a positive value'. Otherwise, the program prompts a product name, adds a new product entry with the user input, and displays the dictionary.

An example run is show here (input underlined):

```
{'A020': [8.0, '137 Degree Real Almond Milk'], 'A121': [4.95, 'UFC
Velvet Almond Milk'], 'B123': [3.15, 'Wholemeal Bread'], 'B138':
[5.95, 'Fruit and Nut Bread']}
Enter code of product: A121
Enter price of product: 5.15
Updating entry A121 with new price 5.15
{'A020': [8.0, '137 Degree Real Almond Milk'], 'A121': [5.15, 'UFC
Velvet Almond Milk'], 'B123': [3.15, 'Wholemeal Bread'], 'B138':
[5.95, 'Fruit and Nut Bread']}
Enter code of product: A122
Enter price of product: 10.95
Enter name of product: Deli Prata
Adding entry A122, 10.95, Deli Prata into products
{'A020': [8.0, '137 Degree Real Almond Milk'], 'A121': [5.15, 'UFC
Velvet Almond Milk'], 'B123': [3.15, 'Wholemeal Bread'], 'B138':
[5.95, 'Fruit and Nut Bread'], 'A122': [10.95, 'Deli Prata']}
Enter name of product code: C111
Enter price of product: 0
Price must be a positive value
{'A020': [8.0, '137 Degree Real Almond Milk'], 'A121': [5.15, 'UFC
Velvet Almond Milk'], 'B123': [3.15, 'Wholemeal Bread'], 'B138':
[5.95, 'Fruit and Nut Bread'], 'A122': [10.95, 'Deli Prata']}
Enter name of product code: A121
Enter price of product: -3
Price must be a positive value
{'A020': [8.0, '137 Degree Real Almond Milk'], 'A121': [5.15, 'UFC
Velvet Almond Milk'], 'B123': [3.15, 'Wholemeal Bread'], 'B138':
[5.95, 'Fruit and Nut Bread'], 'A122': [10.95, 'Deli Prata']}
Enter name of product code:
Program ends
```

(15 marks)

----- END OF PAPER -----